



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ2154

OBJECT-ORIENTED PROGRAMMING

SECTION 03

LECTURER NAME: DR MUHAMMAD KHATIBSYARBINI

MINI PROJECT: VEHICLE RENTAL SYSTEM

NAME	MATRIC NO
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160

Table of Content

1.0 Vehicle Rental System Project Description	3
2.0 Implementation of Each Chapters 1-5	4
3.0 Output Screenshots	8
4.0 Vehicle Rental System UML Diagram	11
5.0 Conclusion	13

1.0 Vehicle Rental System Project Description

This project is a console-based Java application that simulates a basic vehicle rental management system. The system allows users to manage the vehicle inventory, customer registration, facilitate vehicle rental and handling the vehicle rental and return processes. The key features of the system include:

Vehicle Management:

- Add new cars and motorcycles with specific attributes (make, model, year, rental rate, registration number, number of doors/fuel type for cars, engine displacement/helmet provision for motorcycles).
- Remove existing vehicles from the fleet, with checks to prevent removal if a vehicle is currently rented.
- Display a comprehensive list of all vehicles in the system.
- List only the vehicles currently available for rent.

Customer Management:

- Register new customers by capturing their name and contact information, assigning a unique customer ID.
- Display a list of all registered customers.

Rental Transaction Management:

- Facilitate vehicle rentals by associating a customer with an available vehicle for a specified duration, calculating the total rental cost.
- Handle vehicle returns, marking the vehicle as available again and removing the rental record.
- Display all active rental transactions.

2.0 Implementation of Each Chapters 1-5

This section consists of the code snippets where the chapters are implemented in our system and explanation of each.

Chapter 5: Vectors and Array List

Arraylist is used to store a list of vehicles for renting, customers registered under corporation and the rentals record.

```
private ArrayList<Vehicle> vehicles; // Stores all vehicles
private ArrayList<Customer> customers; // Stores all registered customers
private ArrayList<Rental> rentals; // Stores all active/past rentals
```

Arraylist allows us to store an unlimited number of records since it's dynamic storage. It also allows dynamic resizing of the list when vehicles are added/removed and supports looping, filtering like available vehicles, and operations like .remove(), .get().

Chapter 6: Class Relationship

Aggregation is used in Vehicle Rental System class. They can exist independently.

```
class VehicleRentalSystem {
    private ArrayList<Vehicle> vehicles; // Stores all vehicles
    private ArrayList<Customer> customers; // Stores all registered customers
    private ArrayList<Rental> rentals; // Stores all active/past rentals
    private int nextCustomerId = 1001; // Simple ID generator
    private int nextRentalId = 1;
```

Association is used in Rental class. Rent cannot exist independently without a customer and vehicle created first.

```
class Rental {
    private String rentalId; // Unique ID for the rental
    private Customer customer; // Association: A Rental has a Customer
    private Vehicle vehicle; // Association: A Rental has a Vehicle
    private String startDate; // For simplicity, using String for dates
    private String endDate; // In a real system, use java.time.LocalDate
    private int rentalDays;
    private double totalCost;
```

This makes sure that rental can only be made if there is a vehicle and customer registered in the system.

Chapter 7: Inheritance

This is used in vehicle class. Vehicle is a superclass with common attributes. Car and Motorcycle are subclasses that inherit from it and extend with extra features:

- Car has numberOfDoors.
- Motorcycle has engineCapacity

```
class Vehicle {
    // Common attributes for all vehicles
    protected String make;
    protected String model;
    protected int year;
    protected double rentalRatePerDay;
    protected boolean isAvailable; // Tracks availability for rental
    protected String registrationNumber; // Unique identifier for each vehicle
}
```

```
class Car extends Vehicle {
    private int numberOfDoors;
    private String fuelType; // e.g., "Petrol", "Diesel", "Electric"
}
```

```
class Motorcycle extends Vehicle {
    private int engineDisplacementCC; // Cubic centimeters
    private boolean helmetProvided;
}
```

This application avoids code duplication by reusing shared logic and enables type hierarchy, allowing all vehicle types to be managed uniformly using a Vehicle reference.

Chapter 8: Polymorphism

Car and Motorcycle override calculateRentalCost() to apply different rental logic (discounts or surcharges).

Vehicle class:

```
public double calculateRentalCost(int days) throws IllegalArgumentException {
    // --- Chapter 9: Exception Handling ---
    // Basic validation for rental days
    if (days <= 0) {
        throw new IllegalArgumentException(s:"Rental days must be positive.");
    }
    return rentalRatePerDay * days;
}
```

Car class:

```
@Override
public double calculateRentalCost(int days) throws IllegalArgumentException {
    // Call superclass method for base calculation
    double baseCost = super.calculateRentalCost(days);
    // Add a small premium for cars (example of polymorphism)
    return baseCost + (days * 5.0); // $5 extra per day for cars
}
```

Motorcycle class:

```
@Override
public double calculateRentalCost(int days) throws IllegalArgumentException {
    double baseCost = super.calculateRentalCost(days);
    // Add a different premium or discount for motorcycles
    return baseCost + (days * 2.0); // $2 extra per day for motorcycles
}
```

This allows dynamic behavior depending on object type at runtime. Makes it easy to treat different vehicle types the same way in loops, while still applying their unique logic.

Chapter 9: Exception Handling

Custom exceptions:

- VehicleInUseException
- VehicleNotFoundException

```
class VehicleInUseException extends Exception {
    public VehicleInUseException(String message) {
        super(message);
    }
}

// Custom exception for when a vehicle is not found
class VehicleNotFoundException extends Exception {
    public VehicleNotFoundException(String message) {
        super(message);
    }
}
```

Built-in exceptions:

- InputMismatchException
- IllegalArgumentException

```
try {
    Vehicle vehicleToRemove = findVehicleByRegistrationNumber(regNumber);
    if (vehicleToRemove == null) {
        // Throw custom exception if vehicle not found
        throw new VehicleNotFoundException("Vehicle with registration " + regNumber + " not found.");
    }

    if (!vehicleToRemove.isAvailable()) {
        // Throw custom exception if vehicle is currently rented
        throw new VehicleInUseException("Vehicle " + regNumber + " is currently rented and cannot be removed.");
    }

    vehicles.remove(vehicleToRemove);
    System.out.println("Vehicle " + regNumber + " successfully removed.");
} catch (VehicleNotFoundException | VehicleInUseException e) {
    System.err.println("Removal Error: " + e.getMessage());
} catch (Exception e) {
    System.err.println("An unexpected error occurred during vehicle removal: " + e.getMessage());
}
```

Try-catch blocks are used in the menu-driven UI to handle incorrect input and display friendly error messages. This prevents the program from crashing due to bad user input or invalid operations, helps maintain data integrity (e.g., blocking removal of rented vehicles) and improves user experience with clear error feedback.

3.0 Output Screenshots

Menu Option For Vehicle Rental System:

```
--- Vehicle Rental System Menu ---
1. Display All Vehicles
2. Display Available Vehicles
3. Display All Customers
4. Register New Customer
5. Rent Vehicle
6. Return Vehicle
7. Display All Rentals
8. Add New Vehicle
9. Remove Vehicle
0. Exit
Enter your choice:
```

Users need to choose the service offered by the system as listed in the menu by entering the index number of the service chosen. For example, if the user wants to display all vehicles, enter 1, vice versa for other choices.

Display All Vehicle:

```
--- All Vehicles ---
1.Car - Registration: ABC-123, Make: Toyota, Model: Camry, Year: 2022, Rate: $50.00/day, Available: Yes, Doors: 4, Fuel: Petrol
2.Motorcycle - Registration: MOTO-456, Make: Honda, Model: CBR500R, Year: 2021, Rate: $35.00/day, Available: Yes, Engine: 500cc, Helmet Included: No
3.Car - Registration: XYZ-789, Make: Honda, Model: CRV, Year: 2023, Rate: $65.00/day, Available: Yes, Doors: 5, Fuel: Petrol
4.Motorcycle - Registration: BIKE-007, Make: Kawasaki, Model: Ninja 400, Year: 2020, Rate: $30.00/day, Available: Yes, Engine: 400cc, Helmet Included: No
```

If users enter 1, they will be met by a list of all vehicles offered by the rental services no matter available or not available.

Display Available Vehicles:

```
--- Available Vehicles ---
1. Car - Registration: ABC-123, Make: Toyota, Model: Camry, Year: 2022, Rate: $50.00/day, Available: Yes, Doors: 4, Fuel: Petrol
2. Motorcycle - Registration: MOTO-456, Make: Honda, Model: CBR500R, Year: 2021, Rate: $35.00/day, Available: Yes, Engine: 500cc, Helmet Included: No
3. Car - Registration: XYZ-789, Make: Honda, Model: CRV, Year: 2023, Rate: $65.00/day, Available: Yes, Doors: 5, Fuel: Petrol
4. Motorcycle - Registration: BIKE-007, Make: Kawasaki, Model: Ninja 400, Year: 2020, Rate: $30.00/day, Available: Yes, Engine: 400cc, Helmet Included: No
-----
```

If users enter 2, they will see a list of vehicles only available for rent.

Display All Customers:

```
--- All Customers ---  
  
Customer ID: CUST1001, Name: Alice Smith, Contact: 123-456-7890  
  
Customer ID: CUST1002, Name: Bob Johnson, Contact: 098-765-4321  
  
-----
```

If users enter 3, they will see a list of all customers registered under the rental service as well as their details.

Register New Customer:

```
Enter customer name: Rohan  
Enter contact number: 017939929929  
New customer registered:  
Customer ID: CUST1003, Name: Rohan, Contact: 017939929929
```

If users enter 4, they can register new customers by inserting all the required information. Once the customer is registered, they will obtain their customer id, example: CUST1003.

Rent Vehicle:

```
Enter Customer ID: CUST1003  
Enter Vehicle Registration Number: abc-123  
Enter Rental Days: 3  
Enter Start Date (YYYY-MM-DD): 27-06-2025  
Enter End Date (YYYY-MM-DD): 30-06-2025  
  
--- Rental Successful! ---  
Rental ID: RENT1  
Customer: Rohan (CUST1003)  
Vehicle: Toyota Camry (ABC-123)  
Period: 27-06-2025 to 30-06-2025 (3 days)  
Total Cost: $165.00  
  
-----
```

When users enter 5, users can rent a vehicle by inserting the required information. Once the form is filled, a receipt containing all the rent information will be shown on the screen along with the rental ID.

Return Vehicle:

```
Enter Rental ID to return: rent1

--- Vehicle Returned Successfully! ---
Rental ID: rent1 for ABC-123 returned.
-----
```

When users enter 6, they can return their rented vehicle by entering the rental ID.

Display All Rentals:

```
--- Active Rentals ---
Rental ID: RENT2
  Customer: Alice Smith (CUST1001)
  Vehicle: Toyota Camry (ABC-123)
  Period: 20-06-2025 to 21-06-2025 (2 days)
  Total Cost: $110.00
-----
```

When users enter 7, they will be shown a list of all the active rentals and their information.

Add New Vehicle:

```
--- Add New Vehicle ---
Enter Vehicle Type (Car/Motorcycle): motorcycle
Enter Maker: kriss
Enter Model: y12
Enter Year: 2019
Enter Rental Rate Per Day: 20
Enter Registration Number: mdr2345
Enter Engine Displacement (CC): 150
Has Helmet (true/false): true
Vehicle added: mdr2345
```

When users enter 8, they can add a new vehicle under the rental services by entering respective information about the vehicle.

Remove Vehicle:

```
Enter Registration Number of vehicle to remove: msd2344
Vehicle msd2344 successfully removed.
```

When users enter 9, they can remove the vehicle from service by entering its registration number,

Exit:

```
Exiting Vehicle Rental System. Goodbye!
```

If the user wants to exit the system, they just need to enter 0 in the menu

4.0 Vehicle Rental System UML Diagram

This section consists of the UML diagram of the rental system as well as the explanation.



The UML diagram shown above represents the detail of the classes, attributes, methods and relationship between different classes of the code. Detailed explanation for each class is as follows.

Class:

1. Vehicle class (Parent class)
 - The class acts as the parent class for more specific vehicle types where the common properties and behaviors are shared by all vehicles in the rental.
2. Car class (Sub class)
 - The class represent the specific type of vehicle (car type)
3. Motorcycle (Sub class)
 - The class represent the specific type of vehicle (motorcycle type)
4. Customer class
 - The class represent the user/person that rents the vehicles
5. Rental class
 - The class represent the single rental transaction that link a customer with a specific vehicle for each different rental period
6. VehicleRentalSystem class
 - The class serves as the central class that manages all operations of the rental system where it holds collections of vehicles, customers and rentals.

Relationship:

- **Inheritance**
 - Car: Car is a type of Vehicle.
 - Motorcycle: Motorcycle is a type of Vehicle.
- **Association**
 - Customer: A Rental is associated with exactly one Customer.
 - Vehicle: A Rental is associated with exactly one Vehicle.
- **Aggregation**
 - **Vehicle:** A VehicleRentalSystem aggregates zero or more Vehicle objects. The vehicles can exist independently of the system.
 - **Customer:** A VehicleRentalSystem aggregates zero or more Customer objects. Customers can exist independently.
 - **Rental:** A VehicleRentalSystem aggregates zero or more Rental objects. Rentals can exist independently

5.0 Conclusion

Last but not least, we can conclude that this Vehicle Rental System project is a well-designed and functional console-based Java application that effectively simulates a basic vehicle rental management system. It comprehensively demonstrates key object-oriented programming principles, utilizing ArrayLists for dynamic data storage of vehicles, customers, and rentals. The system employs clear class relationships, specifically aggregation for managing independent entities like vehicles and customers, and association to ensure rental transactions are linked to existing customers and vehicles. Inheritance is robustly implemented with Vehicle as the superclass for Car and Motorcycle, promoting code reusability and a logical hierarchy, while polymorphism allows for distinct rental cost calculations across different vehicle types. Furthermore, the project incorporates thorough exception handling using both custom and built-in exceptions, which significantly enhances the application's stability and user experience by gracefully managing invalid inputs and operations. Overall, this project effectively applies fundamental OOP concepts to create a practical, stable, and user-friendly vehicle rental system.